DAX Masterclass

Measures and Time Intelligence in Power BI

Practice dataset: Blinkit (sales, stores, products) | Modeling tab > New measure

What is DAX?

DAX = Data Analysis Expressions. It is the formula language of Power BI. We use it to create **calculated columns** , **measures** and **calculated tables** . Unlike Excel, DAX does not work on single cells. It works on the whole data model: tables, columns and relationships.

What we will build today

0 Setup : relationships, DateTable, Measures table

1 Measure vs Calculated Column (filter context vs row context)

2 Basic measures : SUM, AVERAGE, DISTINCTCOUNT, DIVIDE

3 CALCULATE : the most important DAX function

4 FILTER inside CALCULATE for custom conditions

5 ALL and ALLEXCEPT : remove filters, % of total

6 Time intelligence : TOTALYTD, DATESYTD, SAMEPERIODLASTYEAR, DATEADD

7 Variables : VAR ... RETURN

8 Common measures : MoM growth, YTD sales, running total, % of total

3 golden rules of DAX

- Column reference needs the table name: *sales[quantity]*
- Measure reference has NO table name: *[Total Revenue]*
- Use *DIVIDE(a, b)* instead of *a / b* . DIVIDE does not break when b is zero.

Our data model

We have 3 CSV tables. Load all of them, check the data types, then build the model. Below are sample rows so you know what each table looks like.

1. sales (FACT table) | 22,330 rows

Every row is one product line inside an order. It stores events and numbers.

order_id	order_date	store_id	product_id	quantity	discount_pct	payment_mode	delivery_time_min
100001	2026-05-28	S008	P094	1	0	UPI	11
100001	2026-05-28	S008	P050	1	0	UPI	11
100001	2026-05-28	S008	P012	2	0	UPI	11
100001	2026-05-28	S008	P087	1	0	UPI	11
100001	2026-05-28	S008	P003	1	0	UPI	11

Other columns: *order_line_id* , *customer_id* , *customer_rating*

2. stores (DIMENSION table) | 46 rows

One row per store. It describes where the order was fulfilled.

store_id	store_name	city	region	store_type	area_sqft
S001	Blinkit Kothrud	Pune	West	Dark Store	1800
S007	Blinkit Andheri	Mumbai	West	Mega Hub	3000
S013	Blinkit Saket	Delhi	North	Dark Store	1800
S021	Blinkit Golf Course Rd	Gurugram	North	Dark Store	1500
S031	Blinkit Banjara Hills	Hyderabad	South	Dark Store	1800

Other columns: *state* , *locality* , *opening_date*

3. products (DIMENSION table) | 95 rows

One row per product. It describes what was sold, with price and cost.

product_id	product_name	category	sub_category	mrp	cost_price
P001	Amul Taaza Milk 500ml	Dairy & Eggs	Milk	29	25.53
P006	Mother Dairy Dahi 1kg	Dairy & Eggs	Curd & Paneer	85	76.22
P015	Bingo Mad Angles 72g	Snacks & Munchies	Chips	20	14.72
P029	Pepsi 750ml	Beverages	Soft Drinks	40	34.4
P041	Sting Energy 250ml	Beverages	Energy Drinks	20	17.05

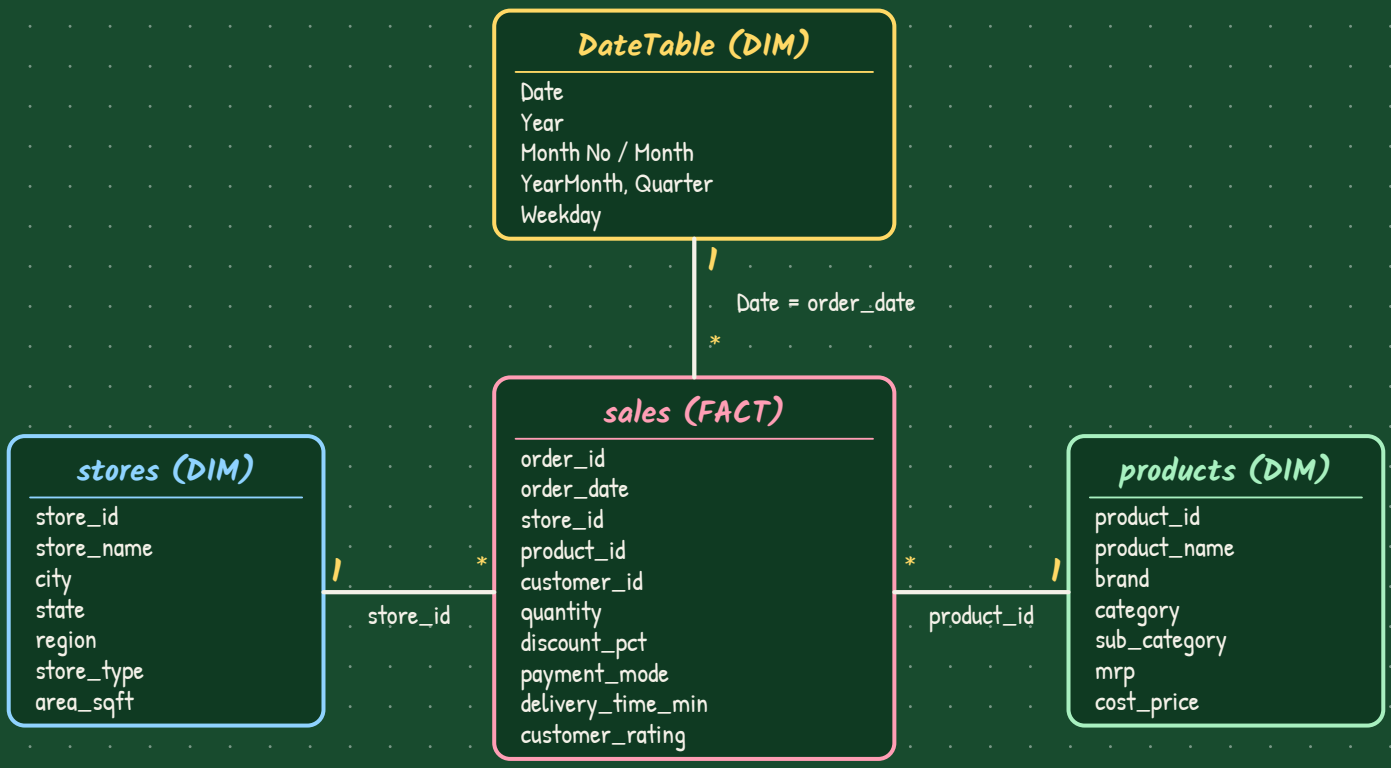
Other columns: *brand* , *is_perishable*

Remember

Fact table = what happened (numbers, events, many rows). ***Dimension table*** = description (who, where, what). A fact table connected to dimension tables is called a ***star schema*** .

0 Step 0: Setup the model

Star schema we will build



1 = One side (unique values). * = Many side (values repeat). All filters flow from dimensions to the fact table.

Build it step by step

0.1 Relationships. Left edge > Model view (3rd icon). Drag *store_id* from stores onto *store_id* in sales. Drag *product_id* from products onto *product_id* in sales. Cardinality: Many to One, direction: Single.

How to create any measure

Select the *Measures* table in the Data pane > Modeling tab > ***New measure*** > delete the default text > paste the formula > press Enter. One formula at a time.

0.2 Create DateTable. Report view > Modeling tab > New table > paste the code below > Enter.

New table

```
DateTable =
ADDCOLUMNS(
    CALENDAR(MIN(sales[order_date]), MAX(sales[order_date])),
    "Year", YEAR([Date]),
    "Month No", MONTH([Date]),
    "Month", FORMAT([Date], "MMM"),
    "YearMonth", FORMAT([Date], "YYYY-MM"),
    "Quarter", "Q" & QUARTER([Date]),
    "Weekday", FORMAT([Date], "ddd")
)
```

Date	Year	Month No	Month	YearMonth	Quarter	Weekday
2025-01-01	2025	1	Jan	2025-01	Q1	Wed
2025-01-02	2025	1	Jan	2025-01	Q1	Thu
2025-01-03	2025	1	Jan	2025-01	Q1	Fri
2025-01-04	2025	1	Jan	2025-01	Q1	Sat

A date table has one row for every date, with no gaps. It is the base for all time intelligence.

0.3 Sort Month. Data view (2nd icon) > click DateTable > click the *Month* column > Column tools > Sort by column > *Month No* . Now months show Jan, Feb, Mar instead of Apr, Aug, Dec.

0.4 Mark as date table. Click DateTable in the Data pane > Table tools > Mark as date table > choose *Date* > OK.

0.5 Connect it. Model view > Home > Manage relationships > New. Top table: sales, column *order_date* . Bottom table: DateTable, column *Date* . Many to One, Single, Active > OK.

0.6 Measures table. Home > Enter data > leave blank > name it *Measures* > Load. Keep all measures here.

Why does Date connect to order_date?

Column names do NOT need to match. A relationship joins two columns that hold the same kind of values. The One side must be unique. Never join on *order_id* : it repeats, so Power BI shows a duplicate values error.

1 Measure vs Calculated Column

	Calculated column	Measure
How it calculates	Row by row	On the visual, for the filters
Context	Row context	Filter context
Where it lives	Stored inside the table	Not stored, calculated on demand
When it updates	On data refresh	When filters or slicers change
Use it for	Row level values	Totals, ratios, KPIs
Where you click	Table tools > New column	Modeling > New measure

Create 4 calculated columns in sales

Data view > click the *sales* table > Table tools > New column . Create them in this order, one at a time.

New column

```
Gross Amount = sales[quantity] * RELATED(products[mrp])
Net Amount = sales[Gross Amount] * (1 - sales[discount_pct] / 100)
Cost Amount = sales[quantity] * RELATED(products[cost_price])
Profit = sales[Net Amount] - sales[Cost Amount]
```

Worked example	quantity	mrp	discount_pct	Gross	Net	cost_price	Profit
one row of sales	2	15	10	30	27.00	11.17	4.66

RELATED goes from the many side (sales) to the one side (products) and picks the matching value for that row. It works because of the relationship.

Now create our first measure. Select the *Measures* table > New measure:

New measure

```
Total Revenue = SUM(sales[Net Amount])
```

Remember

Row context : the formula runs once for every row and stores one answer per row. **Filter context** : the formula runs on whatever rows the visual has filtered (a city, a month, a slicer). The same measure shows a different answer in every cell of a visual.

2 Basic measures

New measures

```
Total Profit = SUM(sales[Profit])
Total Quantity = SUM(sales[quantity])
Total Orders = DISTINCTCOUNT(sales[order_id])
Avg Order Value = DIVIDE([Total Revenue], [Total Orders])
Avg Delivery Time = AVERAGE(sales[delivery_time_min])
Profit Margin % = DIVIDE([Total Profit], [Total Revenue])
```

Function	What it does
SUM(column)	Adds all values of a column
AVERAGE(column)	Mean of a column
DISTINCTCOUNT(column)	Counts unique values. One order has many lines, so we do not use COUNT
DIVIDE(a, b)	Safe division. Returns blank instead of an error when b = 0

Test it: Matrix visual > Rows: *stores[region]* > Values: the measures above. Same measure, different answer per region. That is filter context.

region	Total Revenue	Total Profit	Total Orders	Avg Order Value	Profit Margin %
East	193,423	22,062	464	417	11.4%
North	778,658	85,687	1,911	407	11.0%
South	1,343,789	153,559	3,210	419	11.4%
West	1,416,521	161,496	3,415	415	11.4%
Total	3,732,391	422,805	9,000	415	11.3%

Your numbers should match this table. If they do not, check the relationships.

3 CALCULATE

What is it?

CALCULATE is the most important DAX function. It calculates an expression after ***changing the filter context*** . It can add a filter, replace a filter or remove a filter.

```
CALCULATE( <expression> , <filter 1> , <filter 2> ... )
           what to calculate    which filters to apply
```

New measures

```
UPI Revenue = CALCULATE([Total Revenue], sales[payment_mode] = "UPI")
Pune Revenue = CALCULATE([Total Revenue], stores[city] = "Pune")
Dairy Revenue = CALCULATE([Total Revenue], products[category] = "Dairy & Eggs")
Weekend Revenue = CALCULATE([Total Revenue], DateTable[Weekday] IN {"Sat","Sun"})
```

Test it: Table visual with *stores[region]* , *Total Revenue* and *UPI Revenue* . UPI Revenue is always smaller because CALCULATE added a payment filter on top of the region filter.

region	Total Revenue	UPI Revenue
East	193,423	91,083
North	778,658	403,690
South	1,343,789	647,555
West	1,416,521	694,154

Put *Pune Revenue* next to *stores[region]* : it repeats the same number in every row. The row says East, but CALCULATE replaced the city filter with Pune, so Pune wins.

4 FILTER inside CALCULATE

A simple filter like *sales[payment_mode] = "UPI"* can be written directly inside CALCULATE. For conditions with >= , <= , <> on a measure or for multiple columns, use FILTER . FILTER returns a table that has only the rows that pass your condition.

New measure

```
High Discount Revenue =
CALCULATE(
    [Total Revenue],
    FILTER(sales, sales[discount_pct] >= 15)
)
```

New measures

```
Premium Product Revenue =
CALCULATE([Total Revenue], FILTER(products, products[mrp] >= 100))

Slow Delivery Orders =
CALCULATE([Total Orders], FILTER(sales, sales[delivery_time_min] > 15))
```

- *FILTER(table, condition)* checks the condition row by row and keeps the rows that are TRUE
- Inside CALCULATE the filtered table becomes the new filter context
- FILTER is slower on big tables. Use a plain CALCULATE filter when it is enough.

When to use which

Simple equals filter: write it directly in CALCULATE. Greater than, less than or complex logic: wrap it in FILTER.

5 ALL and ALLEXCEPT

- ALL(table or column) removes all filters from that table or column
- ALLEXCEPT(table, column) removes all filters from the table except the column you name
- Main use: get the grand total as the denominator for % of total

New measures

```
Revenue All Stores = CALCULATE([Total Revenue], ALL(stores))

Revenue % of Total =
DIVIDE([Total Revenue], CALCULATE([Total Revenue], ALL(stores)))

Category % of Total =
DIVIDE([Total Revenue], CALCULATE([Total Revenue], ALL(products)))

Revenue % within City =
DIVIDE([Total Revenue], CALCULATE([Total Revenue], ALLEXCEPT(stores, stores[city])))
```

Formula idea

% of total = this row divided by the grand total. Numerator keeps the filter. Denominator uses ALL to remove it.

region	Total Revenue	Revenue All Stores	Revenue % of Total
East	193,423	3,732,391	5.2%
North	778,658	3,732,391	20.9%
South	1,343,789	3,732,391	36.0%
West	1,416,521	3,732,391	38.0%
Total	3,732,391	3,732,391	100.0%

Test ALLEXCEPT: Table with *stores[city]* , *stores[store_name]* , *Total Revenue* , *Revenue % within City* . The stores of each city add up to 100%. Pune looks like this:

store_name	Total Revenue	Revenue % within City
Blinkit Baner	79,308	15.2%
Blinkit Hadapsar	116,745	22.4%
Blinkit Hinjewadi	103,067	19.7%
Blinkit Kothrud	65,148	12.5%
Blinkit Viman Nagar	72,457	13.9%
Blinkit Wakad	85,448	16.4%
Pune total	522,173	100.0%

ALL(stores) also removes slicer filters on stores, so the % can look "frozen" when you use a city slicer. ALLSELECTED is the next level function for that.

6 Time Intelligence

3 conditions

Time intelligence needs (1) a ***DateTable*** with continuous dates, (2) it must be ***marked as date table*** , (3) it must be ***related*** to sales. In every formula and visual axis use *DateTable[Date]* or *DateTable[YearMonth]* , never *sales[order_date]* .

Function	What it does
TOTALYTD(measure, dates)	Year to date total. Resets every January
DATESYTD(dates)	Dates from Jan 1 till now. Used inside CALCULATE. Same as TOTALYTD
SAMEPERIODLASTYEAR(dates)	Shifts the same period back by exactly 1 year
DATEADD(dates, n, unit)	Shifts dates by n DAY, MONTH, QUARTER or YEAR. n can be negative

New measures

```
Revenue YTD = TOTALYTD([Total Revenue], DateTable[Date])

Revenue YTD (DATESYTD) =
CALCULATE([Total Revenue], DATESYTD(DateTable[Date]))

Revenue LY =
CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(DateTable[Date]))

Revenue Prev Month =
CALCULATE([Total Revenue], DATEADD(DateTable[Date], -1, MONTH))

Revenue Prev Quarter =
CALCULATE([Total Revenue], DATEADD(DateTable[Date], -1, QUARTER))
```

Test it: Table visual with *DateTable[YearMonth]* and the measures. Your 2026 rows should look like this. 2025 rows show blank for Revenue LY because there is no 2024 data.

YearMonth	Total Revenue	Revenue Prev Month	Revenue LY	Revenue YTD
2026-01	174,993	169,787	140,567	174,993
2026-02	161,798	174,993	143,459	336,792
2026-03	223,635	161,798	191,049	560,427
2026-04	195,231	223,635	162,163	755,658

TOTALYTD is the shortcut. DATESYTD with CALCULATE is the long form. Both give the same answer, so show them side by side in class.

7 Variables: VAR ... RETURN

A variable stores a value or a table under a name. You calculate once and reuse the name. Syntax: *VAR name = expression* , then *RETURN* the final result.

Without VAR: repeated and hard to read

```
YoY % No Var =
DIVIDE(
    [Total Revenue] - CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(DateTable[Date])),
    CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(DateTable[Date]))
)
```

With VAR: clean and faster

New measures

```
Revenue YoY % =
VAR CurrentRevenue = [Total Revenue]
VAR LastYearRevenue =
    CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(DateTable[Date]))
RETURN
    DIVIDE(CurrentRevenue - LastYearRevenue, LastYearRevenue)

Revenue MoM % =
VAR CurrentRevenue = [Total Revenue]
VAR PrevMonthRevenue =
    CALCULATE([Total Revenue], DATEADD(DateTable[Date], -1, MONTH))
RETURN
    DIVIDE(CurrentRevenue - PrevMonthRevenue, PrevMonthRevenue)
```

- Readable : each step has a meaningful name
- Faster : the calculation runs once, not twice
- Easy to debug : write *RETURN LastYearRevenue* to check one variable alone
- A variable keeps the filter context of the place where it was defined
- Set the format of both measures to Percentage. Use Shift+Enter for a new line in the formula bar.

8 Common measures

Running total

New measure

```
Running Total Revenue =
CALCULATE(
    [Total Revenue],
    FILTER(
        ALL(DateTable[Date]),
        DateTable[Date] <= MAX(DateTable[Date])
    )
)
```

- *ALL(DateTable[Date])* removes the month filter and opens up all dates
- *MAX(DateTable[Date])* is the last date of the current row (for example, end of March)
- *FILTER* keeps only the dates up to that last date. So we add everything from the start till now.

YearMonth	Total Revenue	Revenue MoM %	Running Total Revenue
2026-01	174,993	3.1%	2,219,136
2026-02	161,798	-7.5%	2,380,934
2026-03	223,635	38.2%	2,604,570
2026-04	195,231	-12.7%	2,799,800

Cheat sheet: what we built

Measure	*Idea*	*Key functions*
Total Revenue	Add up Net Amount	SUM
Profit Margin %	Profit divided by revenue	DIVIDE
UPI Revenue	Revenue for one payment mode	CALCULATE
High Discount Revenue	Revenue where discount >= 15	CALCULATE + FILTER
Revenue % of Total	Row divided by grand total	CALCULATE + ALL
Revenue % within City	Share inside own city	ALLEXCEPT
Revenue YTD	Year start to today	TOTALYTD, DATESYTD
Revenue LY / YoY %	Same period last year	SAMEPERIODLASTYEAR + VAR
Revenue Prev Month / MoM %	Previous month and growth	DATEADD + VAR
Running Total Revenue	Cumulative from the start	CALCULATE + FILTER + ALL

If something goes wrong

- Same number in every row : a relationship is missing. Check Model view.
- Time intelligence shows blank : you used *sales[order_date]* instead of *DateTable[Date]* , or the DateTable is not marked as a date table.
- Column cannot be found : create the calculated columns of Part 1 first, in *sales* .
- Duplicate values error on a relationship : you picked the wrong column. Use *order_date* , not *order_id* .
- Months in wrong order : sort *Month* by *Month No* .